

Assignment-4

Introduction

This project upgrades the SimpleLoader from Assignment 1, replacing its "eager loading" mechanism with a "lazy loading" strategy. This new SimpleSmartLoader implements demand paging, a core concept in modern operating systems.

Instead of loading the entire program into memory at the start, this loader maps and loads nothing upfront. It intentionally triggers a SIGSEGV (segmentation fault) by jumping to the program's virtual entry point, catches this signal, and treats it as a page fault. In response, it maps and loads only the single 4KB page of data or code that was just requested. This process repeats for every new page the program accesses, dramatically improving startup time and reducing memory usage.

Approach

Our implementation is centered around intercepting and handling SIGSEGV signals.

1. Parse Headers: The loader first reads the ELF header and program headers, storing the program header table in a global variable for the signal handler to access.
2. Set "Trap": We use sigaction to register a custom function, `page_fault_handler`, to run whenever a SIGSEGV occurs.
3. Trigger Fault: We jump directly to the program's original virtual entry point (`e_entry`). Since this address isn't mapped, the CPU immediately triggers the SIGSEGV, and the OS passes control to our handler.
4. Handle Fault (`page_fault_handler`): This is the core "loader."
 - Find Segment: It identifies which PT_LOAD segment the `fault_addr` belongs to.
 - Align Address: It finds the start of the 4KB page using the bitmask: `... & ~ (PAGE_SIZE - 1)`.
 - Map Page: It calls `mmap` with `MAP_FIXED` to map exactly one 4KB, zero-filled page at the correct virtual address.
 - Load Data: It checks if the page is:
 - Case 1 (Pure BSS): The page is all zeros. We do nothing, as `MAP_ANONYMOUS` already zeroed it.
 - Case 2 (Pure File): The page is full of code/data. We `lseek` and read 4KB from the ELF file.
 - Case 3 (Straddle): The page is part file-data, part BSS. We `lseek` and read only the data part, leaving the rest as zeros.
5. Report: We count faults and allocations in the handler. We calculate internal fragmentation only when the last page of a segment is faulted, adding the unused space on that page. These totals are printed on exit.

AI usage

We used the AI as a programming assistant for:

- Providing the initial structure for catching SIGSEGV with sigaction and getting the fault_addr.
- help in explaining complex topics like page alignment (~(PAGE_SIZE-1)), the p_filesz vs. p_memszrelationship (BSS), and the "straddle" page logic.
- uintptr_t page_start_addr = (uintptr_t)fault_addr & ~(PAGE_SIZE - 1);
- uintptr_t page_offset = page_start_addr - phdr->p_vaddr;
- off_t file_offset = phdr->p_offset + page_offset;
- sa.sa_flags = SA_SIGINFO;

Workflow

The loader's execution is a simple, repeating loop:

- 1.Loader jumps to an unmapped virtual address (e.g., 0x8048100).
- 2.PAGE FAULT! The OS pauses the program.
- 3.OS calls our page_fault_handler.
- 4.The handler maps the 4KB page (e.g., 0x8048000) and loads its data from the file.
- 5.Handler returns.
- 6.The OS resumes the program, which re-runs the failed instruction—this time succeeding.
- 7.The program runs until it accesses a new unmapped page.
- 8.Go back to Step 2.

Contributions

Somya	2024560	• Designed the 3-case if-else-if logic for data loading (BSS, file, straddle). • Developed the precise internal fragmentation algorithm (detecting the last page). • Created the Makefile and nostdlib test executables (fib.c, sum.c). • Led integration debugging and final testing.
-------	---------	--

Arjun Khatri	2024106	• Implemented the core ELF header parsing and global variable setup. • Wrote the sigaction signal handler registration (setup_signal_handler). • Implemented the page mapping logic using mmap with MAP_FIXED. • Managed the fault and allocation counters for reporting.
--------------	---------	---

References

Lecture 15-20

Github Link

[Github Link](#)